Guía de solución - Caso 2: Sistema de evaluación de prioridad en un hospital (Triage)

Elaborado por: Henry Antonio Mendoza Puerta

Enunciado

El caso solicita clasificar a los pacientes según la urgencia, con reglas distintas por especialidad, aplicando el patrón Strategy:

- Interfaz TriageStrategy con el método int evaluate(Patient patient).

- Estrategias concretas: CardiologyTriageStrategy, TraumaTriageStrategy y PediatricTriageStrategy.

- El sistema asigna la estrategia según el departamento que atiende al paciente.

Cómo aplicamos el patrón Strategy

Las reglas de cada especialidad (cardiología, trauma y pediatría) están en una clase que implementa la interfaz Triaje.

El contexto EvaluadorTriaje recibe una estrategia y evalúa al paciente sin conocer las reglas.

ServicioTriaje elige la estrategia según el departamento.

Si se agrega una especialidad, solo se crea una clase nueva y se registra en el servicio: lo demás no cambia.

En esta solución: TriageStrategy = Triaje, evaluate = evaluar, Patient = Paciente.

Paquetes del proyecto

Paquete	Contenido
model	Clases de datos y enumerados: Departamento y Paciente.
strategy	El patrón: interfaz Triaje, las tres especialidades y el contexto EvaluadorTriaje.
service	Elige la estrategia según el departamento y evalúa al paciente (ServicioTriaje).
view	La ventana (JFrame) en VistaTriaje.
controller	ControladorTriaje: conecta la ventana con el servicio.
(raíz)	Principal: inicia el programa.

Paso a paso

Paso 1. Modelo (Departamento y Paciente)

Enumerado con las especialidades y clase con nombre, edad y nivel de dolor.

Archivo: Departamento.java

```
package com.usil.model;

public enum Departamento {
    CARDIOLOGIA, TRAUMA, PEDIATRIA
}
```

Archivo: Paciente.java

```
package com.usil.model;

public class Paciente {

    private String nombre;
    private int edad;
    private int nivelDolor;

    public Paciente(String nombre, int edad, int nivelDolor) {
        this.nombre = nombre;
        this.edad = edad;
        this.nivelDolor = nivelDolor;
    }

    public String getNombre() {
        return nombre;
    }

    public int getEdad() {
        return edad;
    }

    public int getNivelDolor() {
        return nivelDolor;
    }
}
```

Paso 2. Interfaz de la estrategia (Triaje)

Recibe un paciente y devuelve la prioridad (1 es la más urgente).

Archivo: Triaje.java

```
package com.usil.strategy;

import com.usil.model.Paciente;

/** Devuelve la prioridad: 1 es la más urgente. */
public interface Triaje {

    int evaluar(Paciente paciente);
}
```

Paso 3. Estrategias concretas

Cada especialidad tiene sus reglas según el nivel de dolor. Pediatría: menores de 2 años siempre prioridad 1.

Archivo: TriajeCardiologia.java

```
package com.usil.strategy;

import com.usil.model.Paciente;

public class TriajeCardiologia implements Triaje {

    @Override
    public int evaluar(Paciente paciente) {
        if (paciente.getNivelDolor() >= 8) {
            return 1;
        }
        return paciente.getNivelDolor() >= 5 ? 2 : 3;
    }
}
```

Archivo: TriajeTrauma.java

```
package com.usil.strategy;

import com.usil.model.Paciente;

public class TriajeTrauma implements Triaje {

    @Override
    public int evaluar(Paciente paciente) {
        if (paciente.getNivelDolor() >= 9) {
            return 1;
        }
        if (paciente.getNivelDolor() >= 7) {
            return 2;
        }
        return paciente.getNivelDolor() >= 4 ? 3 : 4;
    }
}
```

Archivo: TriajePediatria.java

```
package com.usil.strategy;

import com.usil.model.Paciente;

/** Los menores de 2 años siempre tienen prioridad 1. */
public class TriajePediatria implements Triaje {

    @Override
    public int evaluar(Paciente paciente) {
        if (paciente.getEdad() < 2) {
            return 1;
        }
        return paciente.getNivelDolor() >= 7 ? 2 : 3;
    }
}
```

Paso 4. Contexto (EvaluadorTriaje)

Usa la estrategia elegida para evaluar al paciente.

Archivo: EvaluadorTriaje.java

```
package com.usil.strategy;

import com.usil.model.Paciente;
```

```
/** Contexto: evalúa al paciente con la estrategia del departamento que lo atiende. */
public class EvaluadorTriaje {

    private Triaje estrategia;

    public void setEstrategia(Triaje estrategia) {
        this.estrategia = estrategia;
    }

    public int evaluar(Paciente paciente) {
        return estrategia.evaluar(paciente);
    }
}
```

Paso 5. Servicio (ServicioTriaje)

Elige la estrategia según el departamento.

Archivo: ServicioTriaje.java

```
package com.usil.service;

import com.usil.model.Departamento;
import com.usil.model.Paciente;
import com.usil.strategy.EvaluadorTriaje;
import com.usil.strategy.Triaje;
import com.usil.strategy.TriajeCardiologia;
import com.usil.strategy.TriajePediatria;
import com.usil.strategy.TriajeTrauma;

public class ServicioTriaje {

    private EvaluadorTriaje evaluador = new EvaluadorTriaje();

    public int evaluar(Paciente paciente, Departamento departamento) {
        Triaje estrategia = switch (departamento) {
            case CARDIOLOGIA -> new TriajeCardiologia();
            case TRAUMA -> new TriajeTrauma();
            case PEDIATRIA -> new TriajePediatria();
        };
        evaluador.setEstrategia(estrategia);
        return evaluador.evaluar(paciente);
    }
}
```

Paso 6. Crear la vista (VistaTriaje)

Clic derecho sobre view, New, JFrame Form, nombre VistaTriaje. En Diseño: JFrame con Border Layout, un JPanel arriba (North) y un JLabel al centro (Center). Agregar al panel estos componentes y renombrarlos (clic derecho, Change Variable Name):

Nombre	Componente	Texto
txtPaciente	JTextField	(vacío)
txtEdad	JTextField	(vacío)
txtDolor	JTextField	(vacío)
cmbDepartamento	JComboBox	(vacío)
btnEvaluar	JButton	Evaluar
lblResultado	JLabel	(vacío)

En Código, fuera del bloque Generated Code, agregar mostrarError y los getters. Archivo completo:

Archivo: VistaTriaje.java

```
package com.usil.view;

import javax.swing.JOptionPane;

public class VistaTriaje extends javax.swing.JFrame {

    public VistaTriaje() {
        initComponents();
        setSize(950, 160);
    }

    public void mostrarError(String message) {
        JOptionPane.showMessageDialog(this, message, "Datos no válidos",
JOptionPane.WARNING_MESSAGE);
    }

    public javax.swing.JTextField getTxtPaciente() {
        return txtPaciente;
    }

    public javax.swing.JTextField getTxtEdad() {
        return txtEdad;
    }

    public javax.swing.JTextField getTxtDolor() {
        return txtDolor;
    }

    public javax.swing.JComboBox<String> getCmbDepartamento() {
        return cmbDepartamento;
    }

    public javax.swing.JButton getBtnEvaluar() {
        return btnEvaluar;
    }

    public javax.swing.JLabel getLblResultado() {
        return lblResultado;
    }
```

```
    /** Inicializa los componentes del formulario. No modificar: lo regenera el editor de
NetBeans (pestaña Diseño). */
    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated
Code">//GEN-BEGIN:initComponents
    private void initComponents() {

        pnlFormulario = new javax.swing.JPanel();
        lblPaciente = new javax.swing.JLabel();
        txtPaciente = new javax.swing.JTextField();
        lblEdad = new javax.swing.JLabel();
        txtEdad = new javax.swing.JTextField();
        lblDolor = new javax.swing.JLabel();
        txtDolor = new javax.swing.JTextField();
        lblDepartamento = new javax.swing.JLabel();
        cmbDepartamento = new javax.swing.JComboBox<>();
        btnEvaluar = new javax.swing.JButton();
        lblResultado = new javax.swing.JLabel();

        setDefaultCloseOperation(javax.swing.WindowConstants.EXIT_ON_CLOSE);
        setTitle("Caso 2 - Triage hospitalario");

        lblPaciente.setText("Paciente:");
        pnlFormulario.add(lblPaciente);

        txtPaciente.setColumns(12);
        pnlFormulario.add(txtPaciente);

        lblEdad.setText("Edad:");
        pnlFormulario.add(lblEdad);

        txtEdad.setColumns(4);
        pnlFormulario.add(txtEdad);

        lblDolor.setText("Dolor (0-10):");
        pnlFormulario.add(lblDolor);

        txtDolor.setColumns(3);
        pnlFormulario.add(txtDolor);

        lblDepartamento.setText("Departamento:");
        pnlFormulario.add(lblDepartamento);

        cmbDepartamento.setModel(new javax.swing.DefaultComboBoxModel<>(new String[] { }));
        pnlFormulario.add(cmbDepartamento);

        btnEvaluar.setText("Evaluar");
        pnlFormulario.add(btnEvaluar);

        lblResultado.setHorizontalAlignment(javax.swing.SwingConstants.CENTER);
        lblResultado.setText(" ");

        getContentPane().add(pnlFormulario, java.awt.BorderLayout.NORTH);
        getContentPane().add(lblResultado, java.awt.BorderLayout.CENTER);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton btnEvaluar;
    private javax.swing.JComboBox<String> cmbDepartamento;
    private javax.swing.JLabel lblDepartamento;
    private javax.swing.JLabel lblDolor;
    private javax.swing.JLabel lblEdad;
    private javax.swing.JLabel lblPaciente;
    private javax.swing.JLabel lblResultado;
    private javax.swing.JPanel pnlFormulario;
    private javax.swing.JTextField txtDolor;
    private javax.swing.JTextField txtEdad;
    private javax.swing.JTextField txtPaciente;
```

```
    // End of variables declaration//GEN-END:variables
}
```

Paso 7. Controlador (ControladorTriaje)

Llena el combo, valida los datos, escucha el botón y muestra la prioridad.

Archivo: ControladorTriaje.java

```
package com.usil.controller;

import com.usil.model.Departamento;
import com.usil.model.Paciente;
import com.usil.service.ServicioTriaje;
import com.usil.view.VistaTriaje;
import java.util.Arrays;

public class ControladorTriaje {

    private VistaTriaje vista;
    private ServicioTriaje servicio;

    public ControladorTriaje(VistaTriaje vista, ServicioTriaje servicio) {
        this.vista = vista;
        this.servicio = servicio;

        // El combo se llena con los valores del enum usando streams.
        Arrays.stream(Departamento.values()).forEach(e ->
vista.getCmbDepartamento().addItem(e.name()));

        vista.getBtnEvaluar().addActionListener(e -> evaluar());
    }

    public void iniciar() {
        vista.setLocationRelativeTo(null);
        vista.setVisible(true);
    }

    private void evaluar() {
        try {
            String nombre = vista.getTxtPaciente().getText().trim();
            int edad = Integer.parseInt(vista.getTxtEdad().getText().trim());
            int dolor = Integer.parseInt(vista.getTxtDolor().getText().trim());
            if (nombre.isEmpty() || edad < 0 || dolor < 0 || dolor > 10) {
                vista.mostrarError("Ingrese el nombre, una edad válida y un dolor entre 0 y
10.");
                return;
            }
            Departamento departamento =
Departamento.values()[vista.getCmbDepartamento().getSelectedIndex()];
            int prioridad = servicio.evaluar(new Paciente(nombre, edad, dolor),
departamento);
            vista.getLblResultado().setText(nombre + " - Prioridad " + prioridad + " (1 = la
más urgente)");
        } catch (NumberFormatException ex) {
            vista.mostrarError("Edad y dolor deben ser números enteros.");
        }
    }
}
```

Paso 8. Crear la clase Principal

Inicia el programa.

Archivo: Principal.java

```
package com.usil;

import com.usil.controller.ControladorTriaje;
import com.usil.service.ServicioTriaje;
import com.usil.view.VistaTriaje;
import javax.swing.SwingUtilities;
import javax.swing.UIManager;

public class Principal {

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception ex) {
                // se usa el aspecto por defecto de Swing
            }
            new ControladorTriaje(new VistaTriaje(), new ServicioTriaje()).iniciar();
        });
    }
}
```

Paso 9. Ejecutar y probar

Clic derecho sobre Principal.java, Run File. Se ingresan los datos de prueba y se presiona el botón.



Figura 1. Ventana inicial.

Paciente	Edad	Dolor	Departamento	Prioridad
Ana Torres	45	9	CARDIOLOGIA	1
Luis Perez	30	5	TRAUMA	3
Sofia Ruiz	1	3	PEDIATRIA	1
Mateo Diaz	8	8	PEDIATRIA	2

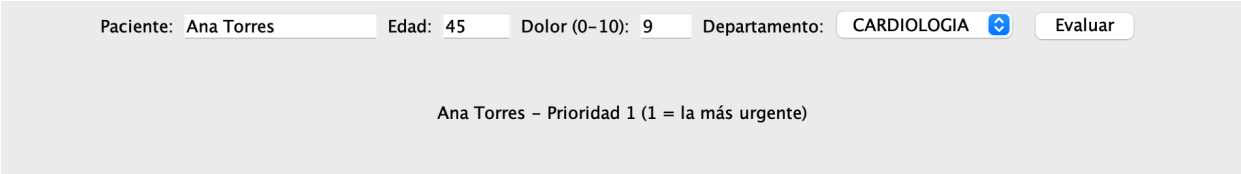


Figura 2. Ana Torres, CARDIOLOGIA: prioridad 1.

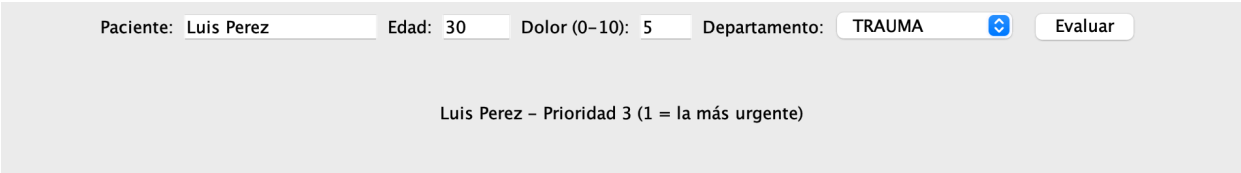


Figura 3. Luis Perez, TRAUMA: prioridad 3.

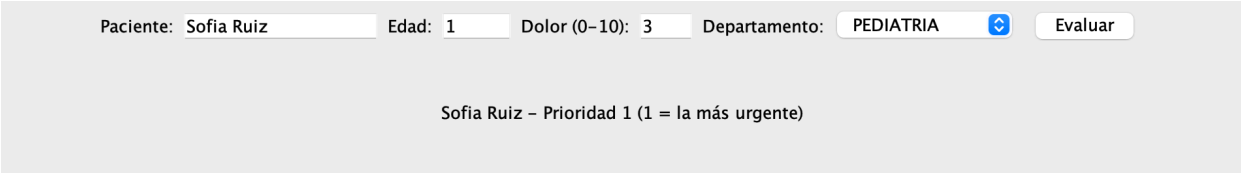


Figura 4. Sofia Ruiz, PEDIATRIA (1 año): prioridad 1.

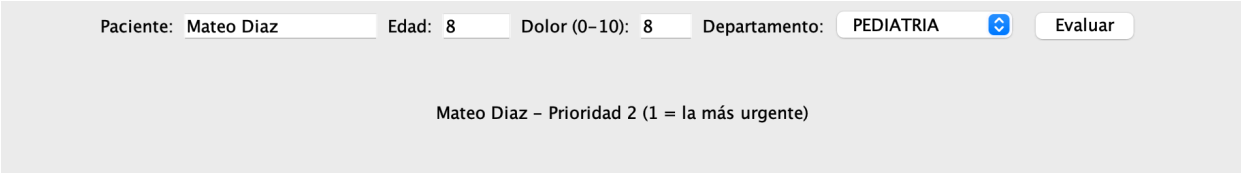


Figura 5. Mateo Diaz, PEDIATRIA: prioridad 2.